MODULE 32 · DOMAIN 4 - OPERATING, MONITORING, SECURING

Cost optimisation for ML

Spot, Savings Plans, right-sizing, tagging, and the idle endpoint problem.

MODULE 32 of 43 DOMAIN 4 - Monitoring WEIGHT 24%

admin@sthithapragna.com

The problem is rarely the hourly rate. It is **paying for hours nothing happened**.

The cost drivers in an ML account

DRIVER	SHAPE
Real-time endpoints	Continuous, per instance-hour. Usually the largest line
Training and tuning jobs	Bursty. Tuning multiplies it by the number of trials
Notebooks and Studio apps	Continuous while running, and easy to forget
Storage	S3, EFS for Studio, Feature Store online store. Accumulates silently
Foundation model tokens	Per use, with no ceiling. Module 25
Always-on supporting services	MLflow tracking servers, NAT gateways, monitoring ingestion

ON THE EXAM

The last row is the one nobody budgets for. An MLflow server and a NAT gateway bill all month and appear on nobody's dashboard.

Cost efficiency, not just cost

- **Cost per prediction** or **cost per resolved ticket** says more than a monthly total.
- A bill that doubles while volume triples is **improving efficiency**, not a problem.
- Track the unit, so growth can be distinguished from waste.
- It also lets you argue for spend, which a raw total never does.
- Without a unit, every cost conversation is about reduction rather than about value.

> ON THE EXAM
>
> Module 17's rule once more: the number worth reporting is the one that moves with the business outcome.

Idle endpoints

- **An idle real-time endpoint bills the same as a busy one.** All night, all weekend.
- The common causes: a proof of concept nobody deleted, a dev endpoint, or a model called rarely.
- Check **invocations against instance-hours**. An endpoint with near-zero invocations is pure waste.
- The fixes are all in this course: serverless, async, batch, multi-model, or scale to zero.
- The simplest fix is deletion. A model can be redeployed from the registry in minutes.

> ON THE EXAM
>
> Every source on SageMaker cost says the same thing: idle endpoints are the most common and most avoidable waste in ML.

How to find idle resources

RESOURCE	HOW TO SPOT IT
Endpoints	Invocations metric near zero against continuous instance-hours
Notebook instances	Running status with no recent activity
Studio apps	Running for days, with the user on holiday
Training clusters	A warm pool retained far longer than the iteration rhythm
Storage	Old checkpoint prefixes nobody has read in months
Provisioned Throughput	Committed capacity with token usage far below it. Module 25

ON THE EXAM

Trusted Advisor and Compute Optimizer surface several of these automatically, which is easier than remembering to look.

Managed Spot Training

- Uses spare EC2 capacity for training, at **up to 90 percent** off the on-demand rate.
- Set **use_spot_instances** true and supply a **checkpoint_s3_uri**.
- **max_wait must be at least max_run**, because it covers training time **plus** waiting for capacity.
- AWS can reclaim the instance with **two minutes' notice**.
- SageMaker AI handles the interruption and resumes, **if checkpointing is configured**.

> ON THE EXAM
>
> The discount is enormous and the condition is specific. Spot without checkpointing means an interrupted job starts again from zero.

Checkpointing is what makes Spot safe

- **Without checkpointing, an interrupted job restarts from the beginning.**
- So a long job on Spot without checkpoints can cost **more** than on demand, and take longer.
- With checkpoints, the job resumes from the last saved state and the interruption costs minutes.
- Checkpoint frequently enough that losing the interval does not matter, and not so often that it dominates.
- Module 14 introduced checkpointing. This is the reason it earns its place.

> ON THE EXAM
>
> The discount is real and conditional. Spot plus checkpointing is a bargain; Spot without it is a gamble with your wall-clock time.

Warm pools

- A **warm pool** keeps training infrastructure provisioned between jobs, for a period you set.
- So the next job **skips provisioning**, which on GPU instances can be several minutes.
- Valuable when iterating: twenty short jobs pay twenty provisioning delays without it.
- **You pay for the retained pool** while it waits. It is a speed purchase, not a saving.
- Set the retention period to your actual iteration rhythm, not to a whole working day.

> ON THE EXAM
>
> Warm pools trade money for wall-clock time, which is the opposite trade to Spot. Know which one you are making.

What actually drives training cost

FACTOR	EFFECT
Instance type	The hourly rate. Often chosen once and never revisited
Instance count	Distributed training multiplies the rate. Module 14
Job duration	Billed by the second, so an inefficient data pipeline costs real money
Number of runs	Experiments, plus every tuning trial. Module 15
Provisioning and idle	Time before training starts, and warm pool retention

ON THE EXAM

Row four is the multiplier people forget. A 50-trial tuning job is 50 training jobs, and the job cost is multiplied, not shared.

Controlling tuning cost

- **Every trial is a full training job.** There is no service charge, and the compute is real.
- **Early stopping** kills unpromising trials, which is the single biggest saving. Module 15.
- **Hyperband** is designed to stop bad trials early, where it can read per-epoch metrics.
- Narrow the search space. A wide range wastes trials exploring values nobody would ship.
- Tune on a **sample** first to find the region, then refine on full data.

> ON THE EXAM
>
> A tuning job is the easiest way to spend a month's training budget in an afternoon, and the easiest to control once you know how.

Spot or on-demand?

SPOT, WHEN

The job can be interrupted

- Checkpointing is implemented.
- Wall-clock time is flexible.
- Experimentation, or scheduled retraining.
- Up to 90 percent cheaper.

ON-DEMAND, WHEN

The job must finish on time

- A hard deadline, or a release depends on it.
- No checkpointing in the training script.
- Capacity for that instance type is scarce.
- Predictable start and finish.

THE TIEBREAKER

Spot trades certainty of timing for a large discount. Ask whether anything downstream depends on when the job finishes.

Spot for other work

- **Processing jobs** can use Spot too, and most data preparation is restartable.
- **Real-time endpoints cannot use Spot.** They must be continuously available.
- Batch transform is already cheap, because instances terminate when the job ends.
- The rule: Spot suits **work that can be restarted**, which excludes anything serving live traffic.
- HyperPod clusters have their own resiliency mechanisms for long-running training.

> ON THE EXAM
>
> Spot is about restartability. Anything a user is waiting on is disqualified before you look at the price.

SageMaker Savings Plans

- Commit to a **consistent hourly spend** for a **one-year or three-year** term, for a lower rate.
- The discount applies **hour by hour**. Usage above your commitment prices at on demand.
- **Usage below the commitment still consumes the full commitment.** Over-committing is money lost.
- Purchased through the Console or the Cost Explorer API, not through the SDK.
- Size it from **measured steady-state usage**, and leave the peaks on demand.

> ON THE EXAM
>
> Size to the floor, not the peak. Committing to your busiest hour means paying for that hour every hour of the year.

What a Savings Plan does and does not cover

COVERED	NOT COVERED
Training jobs	Batch Transform
Processing jobs	Ground Truth
Real-time inference endpoints	Feature Store storage
Studio compute and notebook instances	Storage generally

ON THE EXAM

The shape is what to remember: persistent compute is covered, batch-style and data services are not. Check current coverage before committing.

Savings Plan or Spot?

SAVINGS PLAN

Predictable spend

- A financial commitment, not a technical change.
- Suits steady, continuous usage.
- Covers endpoints, which Spot cannot.
- No interruption risk at all.

SPOT

Interruptible work

- A technical choice, per job.
- Suits bursty, fault-tolerant training.
- Training and processing only.
- Interruption is the price.

THE TIEBREAKER

They are complementary. A steady endpoint fleet on a Savings Plan, and experimental training on Spot, is a normal combination.

Right-sizing

- Match the instance to the workload, which module 22 covered in detail.
- **A GPU that is 20 percent utilised is 80 percent waste**, at the most expensive rate you pay.
- **Inference Recommender** replaces guesswork with a measured recommendation.
- Graviton for CPU inference, and smaller instances after quantization. Module 22.
- Most teams pick an instance type once and never revisit it, even as the model changes.

> ON THE EXAM
>
> This is the only lever that costs nothing to apply and requires no commitment. It is also the one most often skipped.

Estimating cost in advance

- Instance rate times expected duration times instance count. The arithmetic is not hard.
- **Multiply by the number of trials** for tuning, and by the number of experiments planned.
- The AWS Pricing API returns on-demand rates programmatically, if you want this automated.
- Run on a **sample first**, measure, then extrapolate. That is more accurate than any estimate.
- A single misconfigured job can consume a shared monthly budget. Estimating takes two minutes.

> ON THE EXAM
>
> The sample run is the honest estimate: it measures duration rather than guessing it, and duration is the uncertain term.

Deleting things safely

- An endpoint can be deleted and **redeployed from the registry** in minutes. Module 21.
- That makes deletion reversible, which is what makes it an acceptable first response to idleness.
- Check **who calls it** first. An endpoint with low traffic may still have one important caller.
- Deleting a notebook instance loses local files. Commit work to a repository first.
- S3 lifecycle rules delete on a schedule, which avoids the decision entirely.

> ON THE EXAM
>
> Deletion feels irreversible and mostly is not, provided the model is in the registry and the code is in a repository.

Storage costs

- **S3** holds datasets, model artefacts, checkpoints and logs. Every training run adds more.
- **S3 lifecycle policies** expire or archive old artefacts automatically. Set them once.
- **Studio uses EFS**, which is considerably more expensive per gigabyte than S3.
- **Feature Store online storage** is priced for low-latency reads, so it is not a place to keep history.
- A **stopped notebook instance still has an EBS volume**, and that volume still bills.

> ON THE EXAM
>
> Storage never triggers an alarm. It simply grows, and the bill grows with it, until somebody looks.

Data movement and networking

- **A NAT gateway** bills per hour and per gigabyte processed, all month, whether used or not.
- **VPC endpoints** for S3 and other services avoid routing that traffic through NAT. Module 35.
- Cross-Region data transfer costs money. Keep training data in the Region you train in.
- Cross-AZ traffic in distributed training is usually free within a placement group, and not always.
- This line appears under VPC, not under SageMaker, so ML teams rarely see it.

> ON THE EXAM
>
> A NAT gateway is one of the commonest surprise costs in a VPC-isolated ML account, and it never appears on an ML dashboard.

Auto-shutdown for notebooks and apps

- A notebook left running on Friday bills until Monday, producing nothing.
- A **lifecycle configuration** can shut down an idle instance automatically after a set period.
- Studio applications have idle shutdown settings that administrators can enforce.
- An hour of idle tolerance is usually plenty, and it costs a few seconds to restart.
- Relying on people to remember does not work, and it is not a discipline problem.

> ON THE EXAM
>
> Automate it. Every organisation that relies on individuals remembering to stop notebooks pays for the ones who forgot.

Cost allocation tags

- Tag resources with **project, team, environment and owner**, and activate the tags for billing.
- Without tags, the bill is one number and no conversation about it can be specific.
- Tag endpoints, training jobs, notebooks, pipelines and Provisioned Throughput.
- **Tags must be activated as cost allocation tags**, and they apply from activation onward, not retrospectively.
- Enforce with tag policies, or resources created in a hurry will be untagged forever.

> ON THE EXAM
>
> Tagging is the prerequisite for every other cost conversation. Without it, 'who owns this endpoint' has no answer.

Showback and chargeback

- **Showback** reports each team's spend. **Chargeback** bills it to their budget.
- Both need tags, and chargeback needs tags people **trust**, which is a higher bar.
- Module 28's multi-account structure makes this easier: the account is the boundary.
- Shared resources need an allocation rule: split the MLflow server, or absorb it centrally.
- Chargeback changes behaviour. Showback mostly produces a report nobody acts on.

> ON THE EXAM
>
> If nobody's budget is affected, nobody deletes an idle endpoint. That is the honest argument for chargeback.

What each cost service does

SERVICE	WHAT IT IS FOR
Cost Explorer	Analyse historical spend, by service, tag, account and time
AWS Budgets	Set a threshold and alert, or act, when spend approaches it
Cost Anomaly Detection	Learns normal spend and alerts on unusual changes
Trusted Advisor	Recommendations, including idle and underutilised resources
Compute Optimizer	Right-sizing recommendations from actual utilisation
Cost and Usage Report	The full detail, for your own analysis

ON THE EXAM

Cost Explorer is retrospective. Budgets and Anomaly Detection are the ones that tell you before the month ends.

A monthly cost review that works

- **Top five line items.** What are they, and does each still need to exist?
- **Every endpoint:** invocations against instance-hours. Anything near zero gets a conversation.
- **Untagged spend:** how much, and whose is it?
- **Anything new since last month**, which is where surprises live.
- Thirty minutes a month. The alternative is a quarterly panic and an indiscriminate cut.

> ON THE EXAM
>
> A short regular review beats an annual optimisation project, because the waste accumulates continuously.

What not to optimise

- **Anything below the noise floor.** Saving twelve dollars a month costs more in attention than it returns.
- **Development velocity.** A slower feedback loop costs engineer time, which is dearer than compute.
- **Reliability.** Dropping to one endpoint instance saves money and removes redundancy. Module 20.
- Optimise the **top three lines**. The rest is rounding, and it will distract you.
- Reversible optimisations first, and commitments last.

> ON THE EXAM
>
> Cost optimisation has diminishing returns and non-obvious costs. The engineer hours spent are a real line too.

Budgets and anomaly detection

- A **budget** can alert at a percentage of forecast, not only at the threshold itself.
- So you hear about an overrun **while the month is still running**, when it can be acted on.
- Budget actions can apply a policy automatically, which is strong medicine and occasionally right.
- **Anomaly detection** catches the shape of a problem rather than a total: a sudden new line item.
- A monthly invoice is the worst possible cost alert. It arrives when nothing can be changed.

> ON THE EXAM
>
> A forgotten GPU endpoint costs the same whether you find it on day 2 or day 30. Only the alert timing differs.

Cost controls in a pipeline

- Set **max_run** on every training job, so a hung job cannot run for days.
- That single setting has saved more accidental spend than any dashboard.
- Use pipeline **parameters** for instance type and count, so dev runs small and production runs full.
- Use **step caching**, from module 28. A skipped step costs nothing.
- Budget actions can stop resources automatically, which is blunt and occasionally exactly right.

> ON THE EXAM
>
> max_run is the cheapest insurance in the whole course: one parameter, and no job can ever run away overnight.

Cost in the design decision

- Module 12's ladder is a cost ladder as well as a complexity one.
- **A managed AI service is usually cheaper than a custom model**, once you count the building.
- Prompting costs less than retrieval, which costs less than fine-tuning, which costs less than training.
- A smaller model that is good enough beats a larger one you have to justify every month.
- The cheapest system is the one you did not build. Engineer time is the largest cost in most projects.

> ON THE EXAM
>
> Most cost optimisation happens at design time, in a decision nobody labels as a cost decision.

Every cost lever in this course

SITUATION	LEVER
Endpoint idle most of the day	Serverless, async, or scale to zero. Modules 20 and 23
Many models, low traffic each	Multi-model endpoint. Module 21
Nobody waiting for results	Batch transform. No idle charge. Module 20
Training that can be interrupted	Managed Spot Training
Steady, continuous compute	Savings Plan
CPU inference	Graviton. Module 22
Model too large for the instance	Quantization, then a smaller instance. Module 22
High generative token spend	Prompt caching, routing, shorter prompts. Modules 12 and 25
Anything at all	Right-size it first. Module 22

ON THE EXAM

Nine levers, and the first four are about utilisation rather than price. That is where the large savings are.

Where foundation model cost comes from

DRIVER	LEVER
Input tokens	Shorter prompts, smaller top-k, prompt caching. Modules 10 and 12
Output tokens	maxTokens, and asking for concise output. Module 16
Conversation history	Resent every turn. Trim or summarise it. Module 16
Retrieved chunks	numberOfResults is a cost setting too. Module 26
Agent steps	Step count multiplies everything. Modules 25 and 27
Reserved capacity	Provisioned Throughput bills whether used or not. Module 25

ON THE EXAM

Output tokens are usually the dearer half, and agent steps multiply every other row in this table.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Large bill, few predictions	Idle endpoints. Compare invocations to instance-hours
Spot job took longer and cost more	No checkpointing. It restarted after interruption
Savings Plan not reducing the bill	The workload is batch transform, which is not covered
Cost rose with no new workloads	A forgotten notebook, or an endpoint that scaled out and stayed
Cannot say which team spent what	No cost allocation tags, or tags never activated
Storage bill growing steadily	No S3 lifecycle policy on checkpoints and artefacts
Tuning job cost far more than expected	Every trial is a full training job. Module 15

ON THE EXAM

Row four is worth a monthly check: module 23's DisableScaleIn and a forgotten proof of concept look identical on a bill.

Fix utilisation before buying a discount. A discount on waste is still waste.

> A research team runs dozens of training experiments a week. Each job takes two to six hours on GPU instances. Their training script already saves checkpoints every epoch to S3.
>
> There is no deadline on any individual job, and results are reviewed the following day.

What should they change?

A Buy a three-year Savings Plan sized to their average weekly GPU usage

B Enable Managed Spot Training, with max_wait set above max_run

C Move training to batch transform to avoid endpoint charges

D Reduce the number of experiments to control cost

◆ PAUSE AND DECIDE

What should they change?

A Buy a three-year Savings Plan sized to their average weekly GPU usage

Experimental training is bursty, not steady. A commitment sized to an average pays for the quiet weeks in full and still overflows in busy ones.

B Enable Managed Spot Training, with max_wait set above max_run

Checkpointing already exists, timing is flexible, and the work is experimental. Every condition for Spot is satisfied, for up to 90 percent off.

C Move training to batch transform to avoid endpoint charges

Batch transform runs inference over a dataset. It does not train models, and there are no endpoints in this scenario.

D Reduce the number of experiments to control cost

Reduces the research rather than the cost per unit of research. The cheaper option costs up to 90 percent less for the same work.

THE SIGNAL

'Checkpoints every epoch' and 'no deadline' are the two conditions Spot needs, and both are stated outright.

A company runs six real-time inference endpoints continuously. Cost Explorer shows eighteen months of near-flat spend on the same instance families, varying by less than 15 percent month to month.

All endpoints are well utilised, having been sized with Inference Recommender. Finance has approved a commitment.

What should they do?

A Move the endpoints to Managed Spot Training to reduce the rate

B Consolidate all six onto one multi-model endpoint

C Enable scale to zero on all six endpoints

D Buy a SageMaker Savings Plan sized to the measured steady state

◆ PAUSE AND DECIDE

What should they do?

A. Move the endpoints to Managed Spot Training to reduce the rate

Managed Spot Training applies to training jobs. It is not available for real-time inference endpoints.

B. Consolidate all six onto one multi-model endpoint

Multi-model raises utilisation across underused models. These are well utilised, so consolidation would create contention rather than savings.

C. Enable scale to zero on all six endpoints

Scale to zero suits endpoints that are idle. These run continuously, so there is nothing to scale to zero, and waking costs failed requests.

D. Buy a SageMaker Savings Plan sized to the measured steady state

Steady, well-utilised, continuous usage with eighteen months of data and approval to commit. Utilisation is already fixed, so a discount is the remaining lever.

THE SIGNAL

'Well utilised' is the permission slip. It is what makes a commitment sensible rather than locking in waste.

A team's ML spend is dominated by a nightly batch transform job scoring 40 million records, plus a small always-on endpoint. They buy a one-year Savings Plan sized to their total average hourly SageMaker spend.

Three months later, the bill has barely moved, and Cost Explorer shows much of the commitment going unused.

What happened?

A The commitment was sized too low, so most usage still prices at on demand

B Savings Plan discounts take effect only after the first billing year

C Batch Transform is not covered by SageMaker Savings Plans

D The plan was purchased in the wrong Region

◆ PAUSE AND DECIDE

What happened?

A The commitment was sized too low, so most usage still prices at on demand

Under-committing leaves usage at on demand, and it does not leave the commitment unused. The scenario says much of it is going unused.

B Savings Plan discounts take effect only after the first billing year

Invented. The discount applies hour by hour from the moment the plan is active.

C Batch Transform is not covered by SageMaker Savings Plans

They sized the commitment against total spend, most of which is batch transform. The uncovered usage prices at on demand while the commitment is wasted.

D The plan was purchased in the wrong Region

Would produce unused commitment, and nothing in the scenario suggests a Region mismatch. The coverage gap explains it directly.

THE SIGNAL

Two symptoms together identify it: the bill did not fall AND the commitment went unused. Only a coverage gap produces both.

> A team must retrain and redeploy a model before a regulatory deadline in four days. Training takes about eleven hours on a large GPU instance. Their training script does not implement checkpointing.
>
> To reduce cost, an engineer proposes enabling Managed Spot Training for this run.

How should you respond?

A. Reject it. Without checkpointing, an interruption restarts the job, risking the deadline

B. Agree. SageMaker checkpoints automatically when Spot is enabled

C. Agree, but set max_wait equal to max_run to bound the delay

D. Reject it. Spot instances are not available for GPU instance types

◆ PAUSE AND DECIDE

How should you respond?

A Reject it. Without checkpointing, an interruption restarts the job, risking the deadline

An eleven-hour job with no checkpoints restarts from zero on interruption. With a regulatory deadline in four days, that is the wrong risk for a discount.

B Agree. SageMaker checkpoints automatically when Spot is enabled

SageMaker manages interruption and resumption, and the script must write the checkpoints. Enabling Spot does not create them.

C Agree, but set max_wait equal to max_run to bound the delay

max_wait must be at least max_run and covers waiting for capacity too. Setting them equal leaves no room to wait, and the restart risk is untouched.

D Reject it. Spot instances are not available for GPU instance types

Factually wrong. GPU instances are exactly where Spot saves the most. Right conclusion, invented reason.

THE SIGNAL

The right answer is 'implement checkpointing, then use Spot next time'. For this run, the deadline decides it.

What to take from this module

- **Idle endpoints are the most common waste.** Compare invocations to instance-hours.
- **Spot needs checkpointing**, and max_wait at least max_run. Up to 90 percent off training.
- **Savings Plans cover persistent compute, not Batch Transform or storage.** Size to the floor.
- **Tag everything, and activate the tags.** Untagged spend cannot be discussed usefully.
- **Fix utilisation before buying a discount.** A discount on waste is still waste.

> ON THE EXAM
>
> Module 33 takes cost into the generative half: token economics, agent spend, and where FM costs actually come from.